

Alexander Villalva, Geremy Galvez.	Periodo Académico: 2026-02	Carrera: Desarrollo De Software
Asignatura: DevOps	Fecha: 02/06/2026	Código:

Investigación.

investigar y documentar los siguientes conceptos:

1. Control de Versiones

Definición: Es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, permitiendo recuperar versiones específicas en el futuro.

Utilidad: Evita la pérdida de datos, permite revertir errores, facilita el trabajo en equipo sobre un mismo código y mantiene un historial transparente de quién hizo qué cambio y cuándo.

Ejemplo práctico: Un equipo de tres desarrolladores trabaja en una página web. El programador A daña el menú de inicio por error. Gracias al control de versiones, el equipo puede "viajar en el tiempo" y restaurar la versión exacta de la web de la noche anterior sin perder el trabajo de los demás.

Fuente bibliográfica: Chacon, S., & Straub, B. (2014). *Pro Git*. Apress.

2. Git.

Definición: Es un software de control de versiones **de código abierto y distribuido**, creado por Linus Torvalds en 2005. Se ejecuta de forma local en la computadora del usuario.

Utilidad: Permite gestionar el historial de cambios de un proyecto de forma local, rápida y eficiente, sin necesidad de estar conectado a internet de manera obligatoria.

Ejemplo práctico: Instalas Git en tu laptop, abres la terminal dentro de la carpeta de tu proyecto y ejecutas el comando `git init`. A partir de ese momento, Git empieza a rastrear cada modificación que hagas en tus archivos de código.

Fuente bibliográfica: Loeliger, J., & McCullough, M. (2012). *Version Control with Git*. O'Reilly Media.

3. GitHub

Definición: Es una plataforma de alojamiento en la nube que utiliza el sistema de control de versiones Git.

Utilidad: Funciona como una red social y espacio de almacenamiento para desarrolladores, permitiendo respaldar proyectos en internet, colaborar con personas de todo el mundo y gestionar flujos de trabajo.

Ejemplo práctico: Subes el código de tu aplicación a una cuenta de GitHub para que un compañero que vive en otro país pueda descargarlo, revisarlo y ayudarte a programar una nueva funcionalidad.

Fuente bibliográfica: Bell, P., & Beer, B. (2014). *Introducing GitHub: A Non-Technical Guide*. O'Reilly Media.

4. Repositorio Local

Definición: Es la copia del proyecto que reside físicamente en el disco duro de la computadora del desarrollador, gestionada por Git.

Utilidad: Permite trabajar de forma aislada, hacer pruebas, guardar cambios y consultar el historial sin depender de una conexión a internet ni afectar el trabajo de los demás.

Ejemplo práctico: La carpeta `C:/Devops/App.py` en tu computadora, que contiene el código que estás escribiendo y una carpeta oculta llamada `.git` donde se guarda tu historial local.

Fuente bibliográfica: Ponuthorai, Preethi & Loeliger, Jon (2022). *Version Control with Git* (3rd ed.). O'Reilly Media.

5. Repositorio Remoto.

Definición: Es la copia del proyecto que se hospeda en un servidor en internet o en una red centralizada (como GitHub, GitLab o Bitbucket).

Utilidad: Sirve como la "fuente de la verdad" del proyecto. Permite centralizar el código para que todo el equipo pueda sincronizar sus avances y sirve como respaldo de seguridad.

Ejemplo práctico: La dirección URL <https://github.com/Alexander738022/mi-proyecto.git> donde está guardado el código final de la aplicación y a la cual todos los programadores del equipo envían sus aportes.

Fuente bibliográfica: Chacon, S., & Straub, B. (2014). *Pro Git*. Apress.

6. Commit

Definición: Es una confirmación o "captura de pantalla" del estado de los archivos en un momento específico dentro del historial de Git.

Utilidad: Guarda de forma permanente un conjunto de cambios con un mensaje descriptivo, creando un punto de restauración al que se puede volver si algo sale mal.

Ejemplo práctico: Terminas de programar el formulario de inicio de sesión. Ejecutas `git commit -m "Agregando el código de la suma en proyecto"`. Ese avance queda registrado para siempre en el historial.

Fuente bibliográfica: Giffin, D. (2020). *Git Essentials*. Packt Publishing.

7. Branch (Rama)

Definición: Es una línea de desarrollo independiente dentro del mismo repositorio.

Utilidad: Permite desarrollar nuevas funciones, solucionar errores o experimentar de forma aislada sin alterar la línea de código principal y estable (usualmente llamada `main` o `master`).

Ejemplo práctico: Creas una rama llamada `interfaz-oscura`. Programas allí el modo oscuro de tu app sin alterar la versión que los usuarios ya están usando. Si algo falla, la rama principal sigue intacta.

Fuente bibliográfica: Loeliger, J., & McCullough, M. (2012). *Version Control with Git*. O'Reilly Media.

8. Merge.

Definición: Es la operación que une dos o más ramas de Git para combinar sus historiales y códigos.

Utilidad: Permite integrar las funciones o correcciones que se desarrollaron por separado en una rama secundaria dentro de la línea de producción principal.

Ejemplo práctico: Una vez que probaste que la rama `Resta-estudiante-B` funciona perfectamente, te posicionas en la rama `main` y ejecutas `git merge Resta-estudiante-B` para incluir el modo oscuro en la aplicación oficial.

Fuente bibliográfica: Chacon, S., & Straub, B. (2014). *Pro Git*. Apress.

9. Pull Request

Definición: Un Pull Request (PR) es una solicitud para fusionar cambios realizados en una rama hacia otra rama dentro de un repositorio Git. Permite revisar, discutir y aprobar modificaciones antes de integrarlas al proyecto principal.

Utilidad

- Facilita la revisión de código.
- Permite detectar errores antes de integrar cambios.

- Mejora la colaboración entre desarrolladores.
- Mantiene la calidad del software.

Ejemplo práctico: Durante la práctica se creó el Pull Request "feat: agregar función de división controlada", mediante el cual se solicitó integrar la funcionalidad de división desde la rama `feature-dividir` hacia la rama `main`.

Fuente bibliográfica consultada

GitHub. (2025). *About pull requests*. <https://docs.github.com/en/pull-requests>

10. Fork

Definición: Un Fork es una copia personal de un repositorio alojado en GitHub. Permite realizar cambios sin afectar al repositorio original.

Utilidad

- Permite contribuir a proyectos externos.
- Facilita la experimentación sin riesgo.
- Favorece el desarrollo colaborativo.

Ejemplo práctico: Un desarrollador puede realizar un Fork de un proyecto de software libre, implementar mejoras y posteriormente enviar un Pull Request al repositorio original.

Fuente bibliográfica consultada

GitHub. (2025). *Fork a repository*. <https://docs.github.com/en/get-started/quickstart/fork-a-repo>

11. Conflictos de Integración

Definición: Un conflicto de integración ocurre cuando dos ramas modifican la misma línea o sección de un archivo y Git no puede decidir automáticamente qué versión conservar.

Utilidad

- Detecta cambios incompatibles.
- Obliga a revisar manualmente el código.
- Mantiene la consistencia del proyecto.

Ejemplo práctico: Durante la práctica, al fusionar la rama `feature-dividir` con `main`, Git detectó un conflicto en el archivo `calculadora.py`, el cual fue resuelto manualmente antes de completar el Merge.

Fuente bibliográfica consultada

Git SCM. (2025). *Basic Branching and Merging*. <https://git-scm.com/book/en/v2>

12. Integración Continua (Continuous Integration - CI)

Definición: La Integración Continua es una práctica de desarrollo en la que los cambios de código son integrados frecuentemente en un repositorio compartido y verificados mediante pruebas automáticas.

Utilidad

- Detecta errores tempranamente.
- Reduce problemas de integración.
- Mejora la calidad del software.
- Automatiza validaciones.

Ejemplo práctico: Cada vez que un desarrollador realiza un Push al repositorio, se ejecutan pruebas automáticas para verificar que el código funcione correctamente.

Fuente bibliográfica consultada

Fowler, M. (2006). *Continuous Integration*.
<https://martinfowler.com/articles/continuousIntegration.html>

13. Entrega Continua (Continuous Delivery)

Definición: La Entrega Continua es una práctica que mantiene el software siempre listo para ser desplegado mediante procesos automatizados de construcción, pruebas y validación.

Utilidad

- Reduce riesgos durante las entregas.
- Aumenta la velocidad de liberación.
- Mejora la confiabilidad del software.

Ejemplo práctico: Una aplicación web genera automáticamente una versión candidata para producción una vez que todas las pruebas son aprobadas.

Fuente bibliográfica consultada

Atlassian. (2025). *Continuous Delivery*. <https://www.atlassian.com/continuous-delivery>

14. Despliegue Continuo (Continuous Deployment)

Definición: El Despliegue Continuo es una práctica donde todos los cambios que superan las pruebas automatizadas son desplegados automáticamente en producción.

Utilidad

- Elimina tareas manuales de despliegue.
- Acelera la entrega de nuevas funcionalidades.
- Permite responder rápidamente a cambios y correcciones.

Ejemplo práctico: Una actualización aprobada pasa automáticamente desde GitHub hacia un servidor de producción sin intervención humana.

Fuente bibliográfica consultada

Amazon Web Services. (2025). *What is Continuous Deployment?*
<https://aws.amazon.com/devops/continuous-deployment>

15. GitHub Actions

Definición: GitHub Actions es una herramienta integrada en GitHub que permite automatizar tareas como compilación, pruebas, integración y despliegue de aplicaciones.

Utilidad

- Automatiza procesos repetitivos.
- Facilita la implementación de CI/CD.
- Reduce errores humanos.
- Mejora la productividad del equipo.

Ejemplo práctico: Al realizar un Push a GitHub, se ejecutan automáticamente pruebas unitarias para verificar el correcto funcionamiento del código.

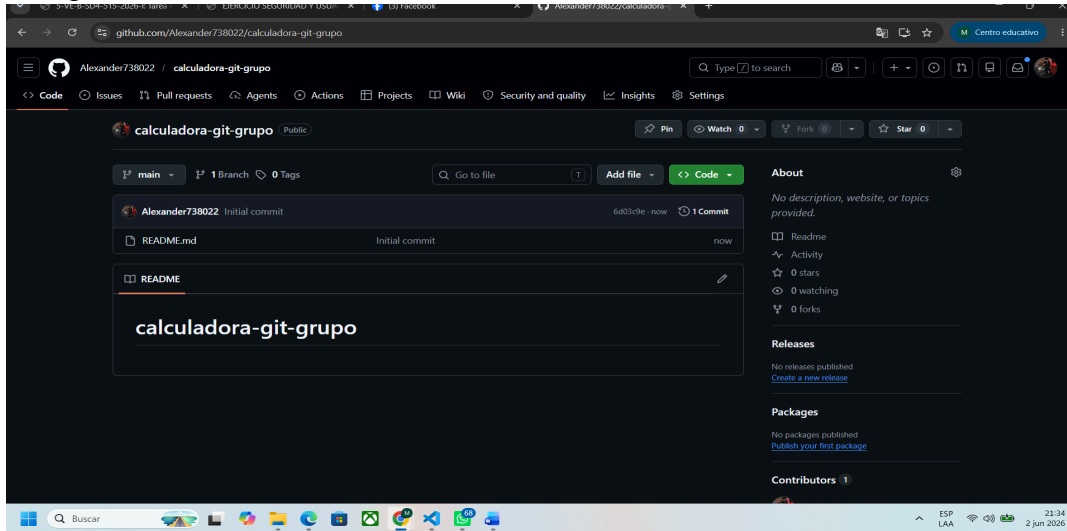
Fuente bibliográfica consultada

GitHub. (2025). *Understanding GitHub Actions*. <https://docs.github.com/en/actions>

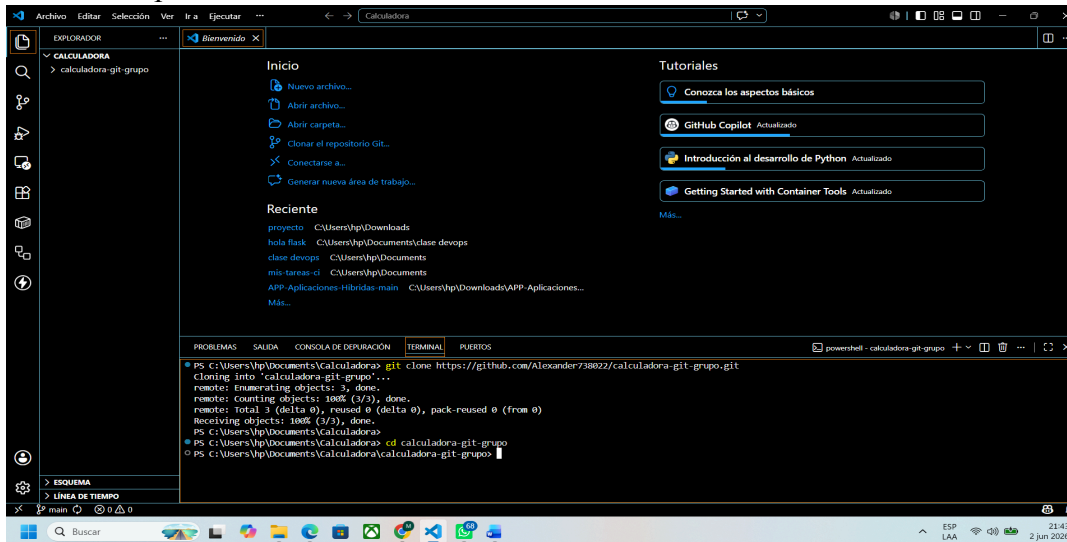
Demostración Práctica.

(Estudiante A)

1: Repositorio web creado.



2: Clonar repositorio de Github.



(Estudiante B)

3. Clonación del repositorio remoto

```
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\CLIENTE\Downloads\proyecto>cd ..

C:\Users\CLIENTE\Downloads>git clone https://github.com/Alexander738022/calculadora-git-grupo.git
Cloning into 'calculadora-git-grupo'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

C:\Users\CLIENTE\Downloads>cd calculadora-git-grupo

C:\Users\CLIENTE\Downloads\calculadora-git-grupo>
```

4. Creación de la rama feature-base

```
C:\Users\CLIENTE\Downloads\calculadora-git-grupo>git checkout -b feature-base
Switched to a new branch 'feature-base'

C:\Users\CLIENTE\Downloads\calculadora-git-grupo>git branch
* feature-base
  main

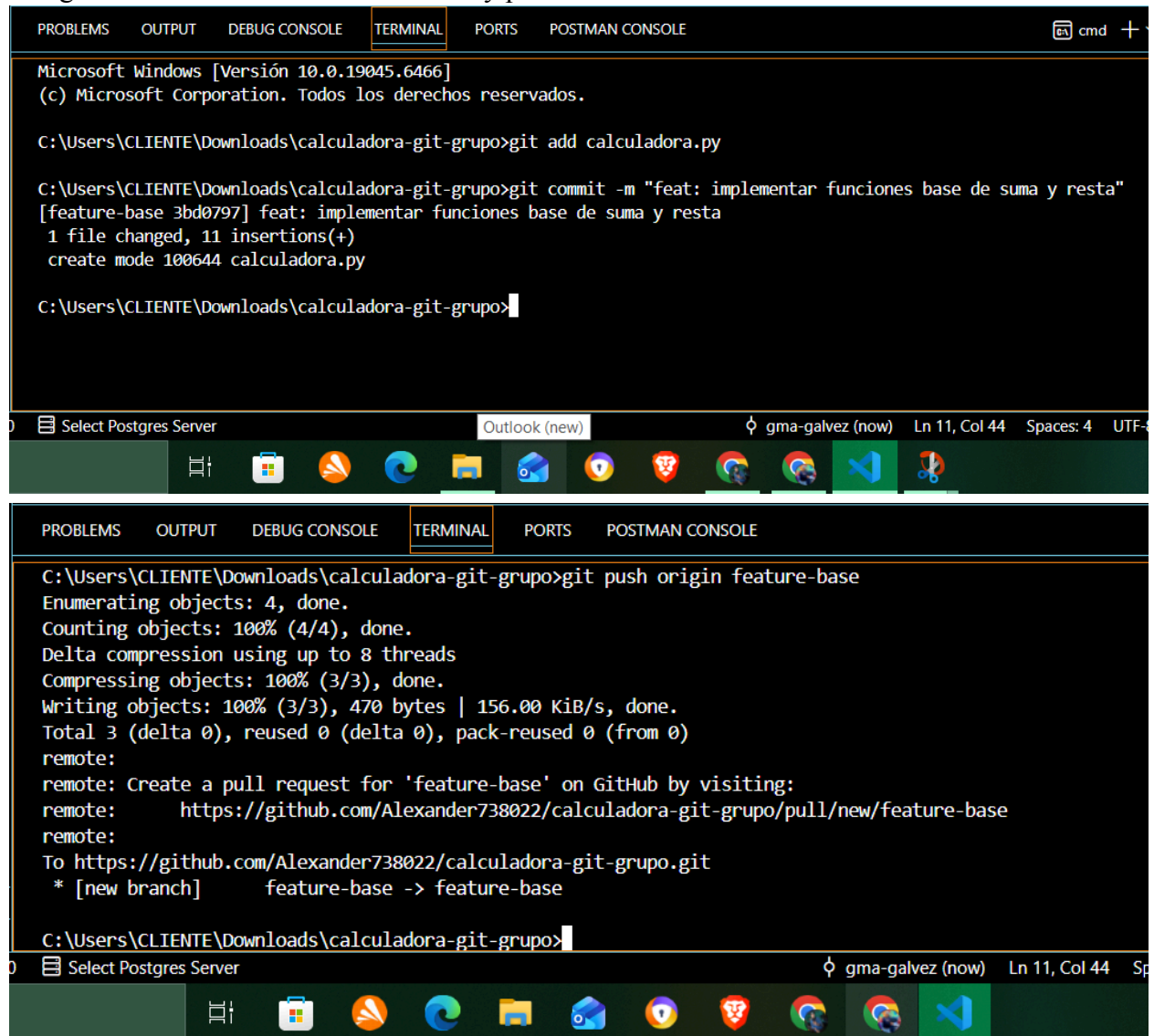
C:\Users\CLIENTE\Downloads\calculadora-git-grupo>
```

5. Implementación de funciones base

```
EXPLORER  calculadora.py U X
CALCULADORA-GIT-GRUPO
  calculadora.py U
  README.md

calculadora.py > ...
1  # Proyecto: Calculadora Colaborativa en Python
2
3  def sumar(a, b):
4      |   return a + b
5
6  def restar(a, b):
7      |   return a - b
8
9  print("--- CALCULADORA ---")
10 print(f"Suma de 5 + 3 = {sumar(5, 3)}")
11 print(f"Resta de 10 - 4 = {restar(10, 4)}")
```


6.Registro de cambios mediante commit y push exitoso



The screenshot shows a Windows terminal window with the following content:

```
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\CLIENTE\Downloads\calculadora-git-grupo>git add calculadora.py

C:\Users\CLIENTE\Downloads\calculadora-git-grupo>git commit -m "feat: implementar funciones base de suma y resta"
[feature-base 3bd0797] feat: implementar funciones base de suma y resta
1 file changed, 11 insertions(+)
create mode 100644 calculadora.py

C:\Users\CLIENTE\Downloads\calculadora-git-grupo>
```

The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), PORTS, and POSTMAN CONSOLE. The status bar at the bottom shows "Select Postgres Server", "Outlook (new)", and "gma-galvez (now) Ln 11, Col 44 Spaces: 4 UTF-8".

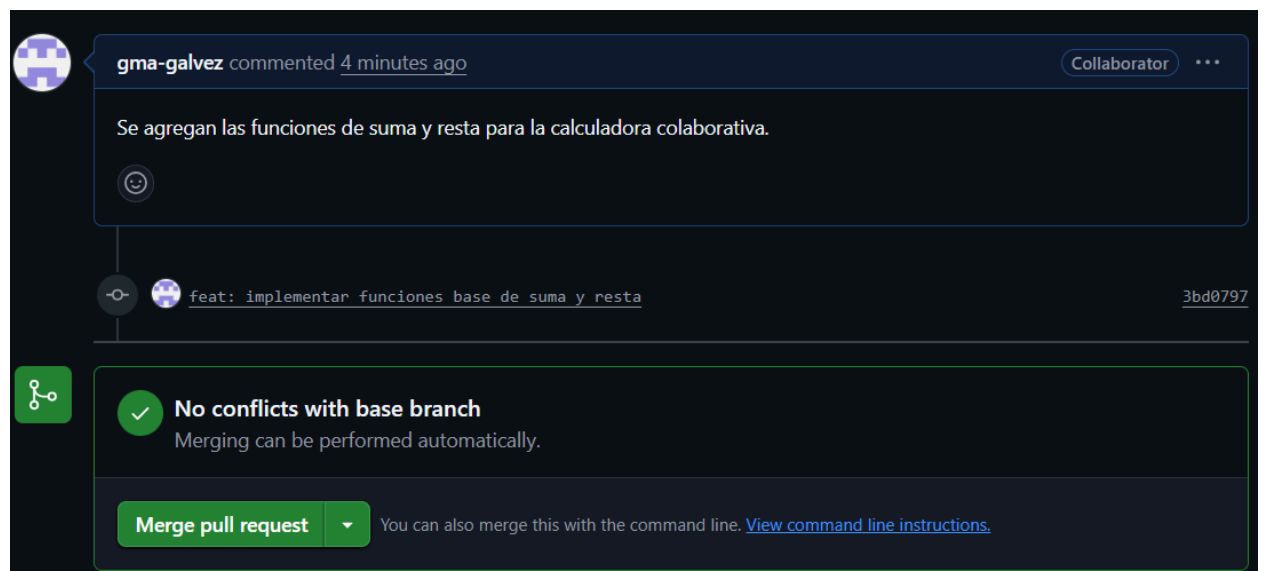
The second screenshot shows the same terminal window with the following content:

```
C:\Users\CLIENTE\Downloads\calculadora-git-grupo>git push origin feature-base
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 470 bytes | 156.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-base' on GitHub by visiting:
remote:   https://github.com/Alexander738022/calculadora-git-grupo/pull/new/feature-base
remote:
To https://github.com/Alexander738022/calculadora-git-grupo.git
 * [new branch]      feature-base -> feature-base

C:\Users\CLIENTE\Downloads\calculadora-git-grupo>
```

The status bar at the bottom shows "Select Postgres Server", "gma-galvez (now) Ln 11, Col 44 Sp".

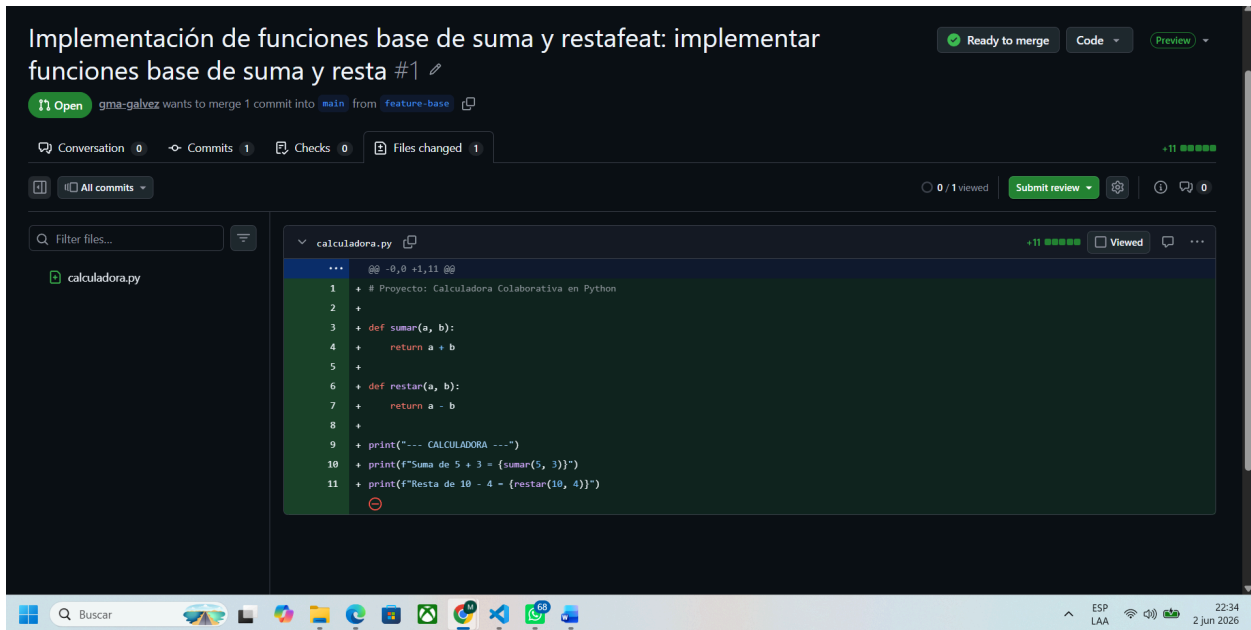
7.Creación del Pull Request



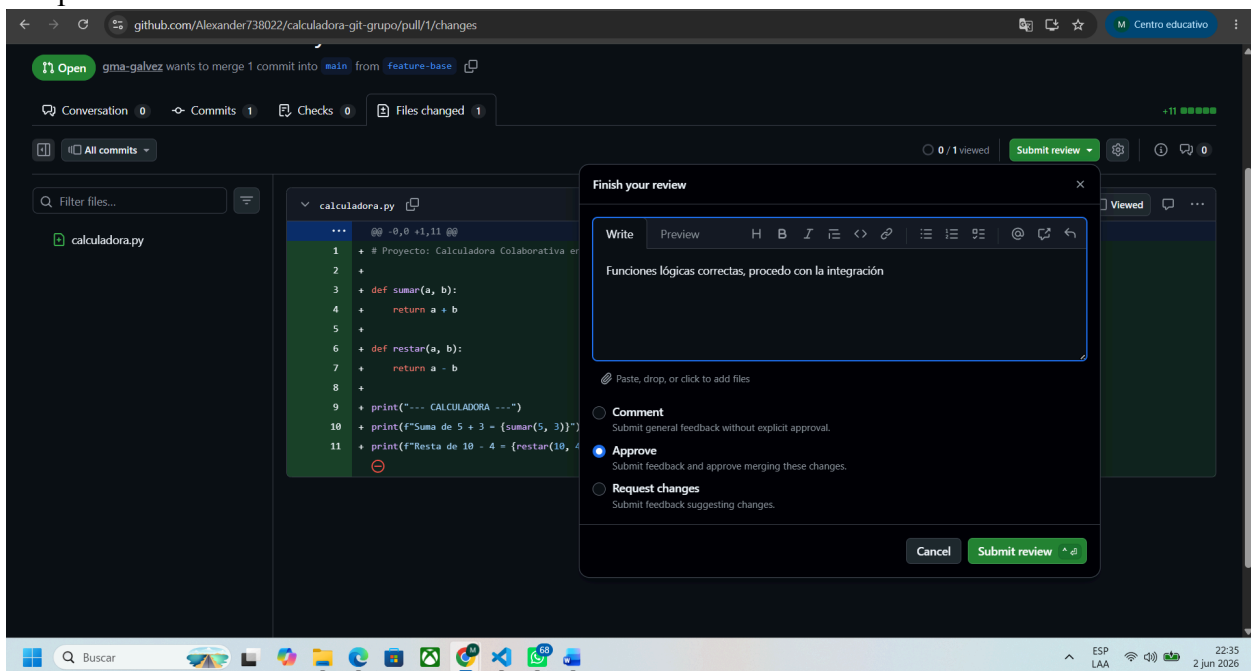
The screenshot shows a GitHub Pull Request interface. At the top, a comment by "gma-galvez" (Collaborator) says "Se agregan las funciones de suma y resta para la calculadora colaborativa." Below the comment is a commit titled "feat: implementar funciones base de suma y resta" with commit hash "3bd0797". At the bottom, a green box indicates "No conflicts with base branch" and "Merging can be performed automatically." There is a "Merge pull request" button and a link to "View command line instructions."

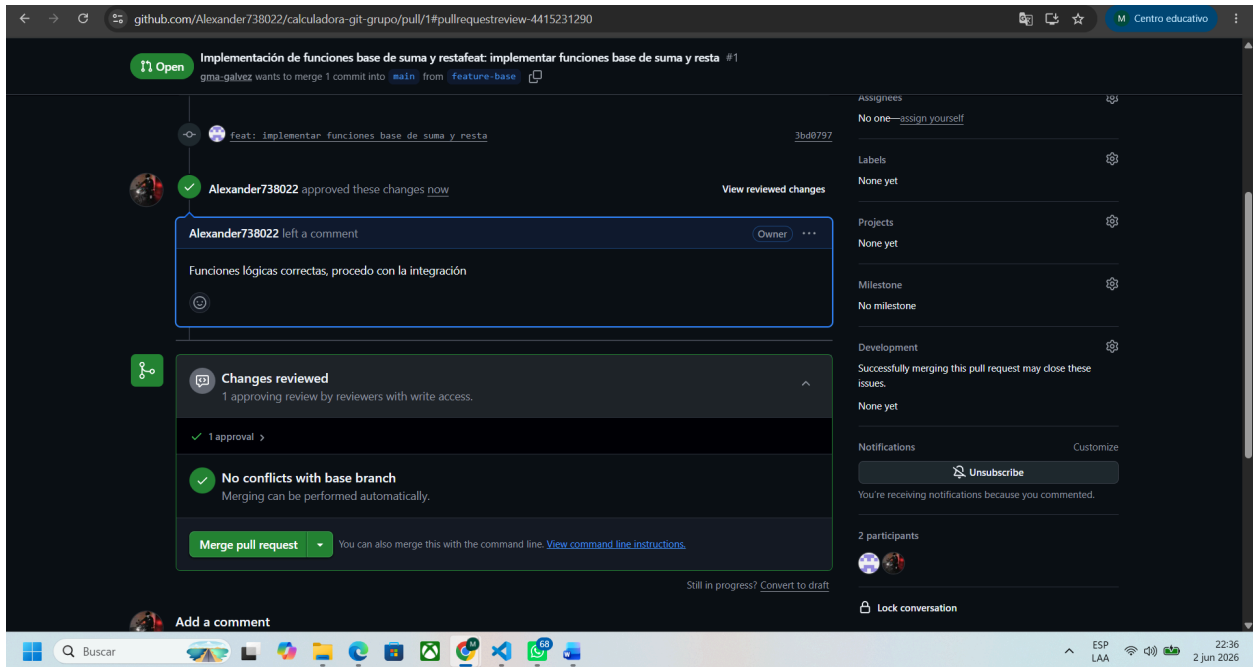
(Estudiante A)

8: Revisión de Código y Primer Merge

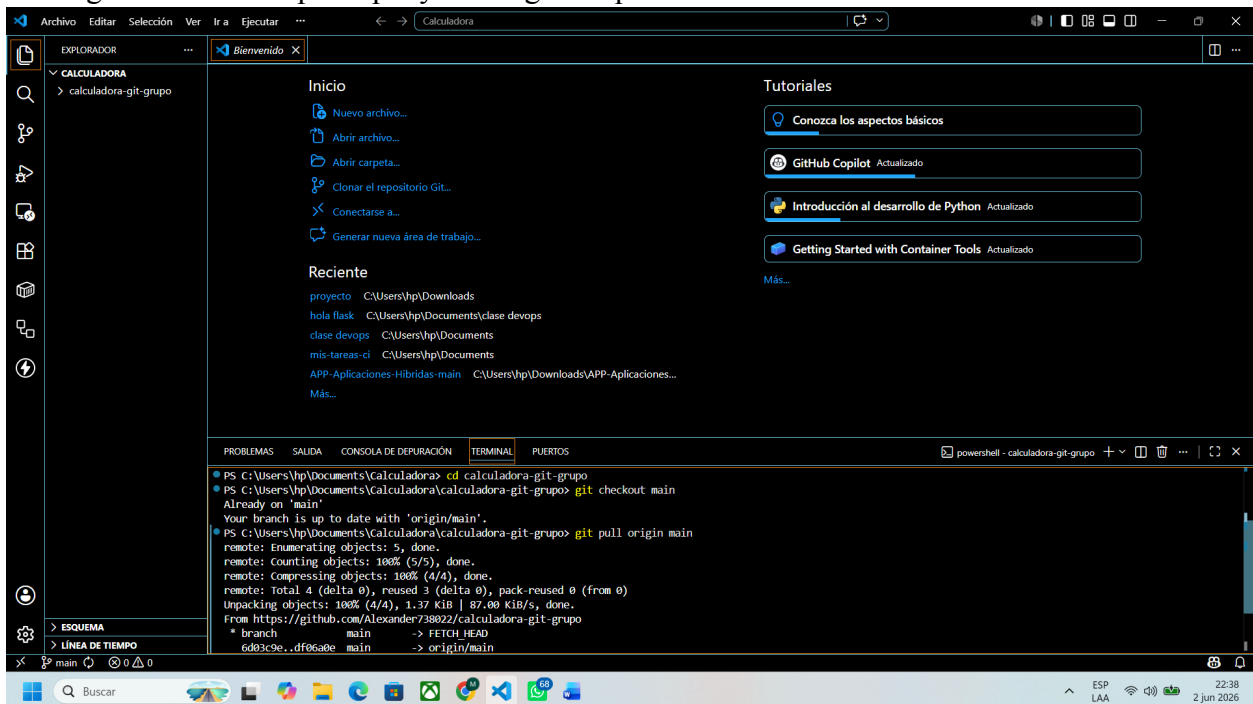


9: Aprobación del PR en la interfaz web.

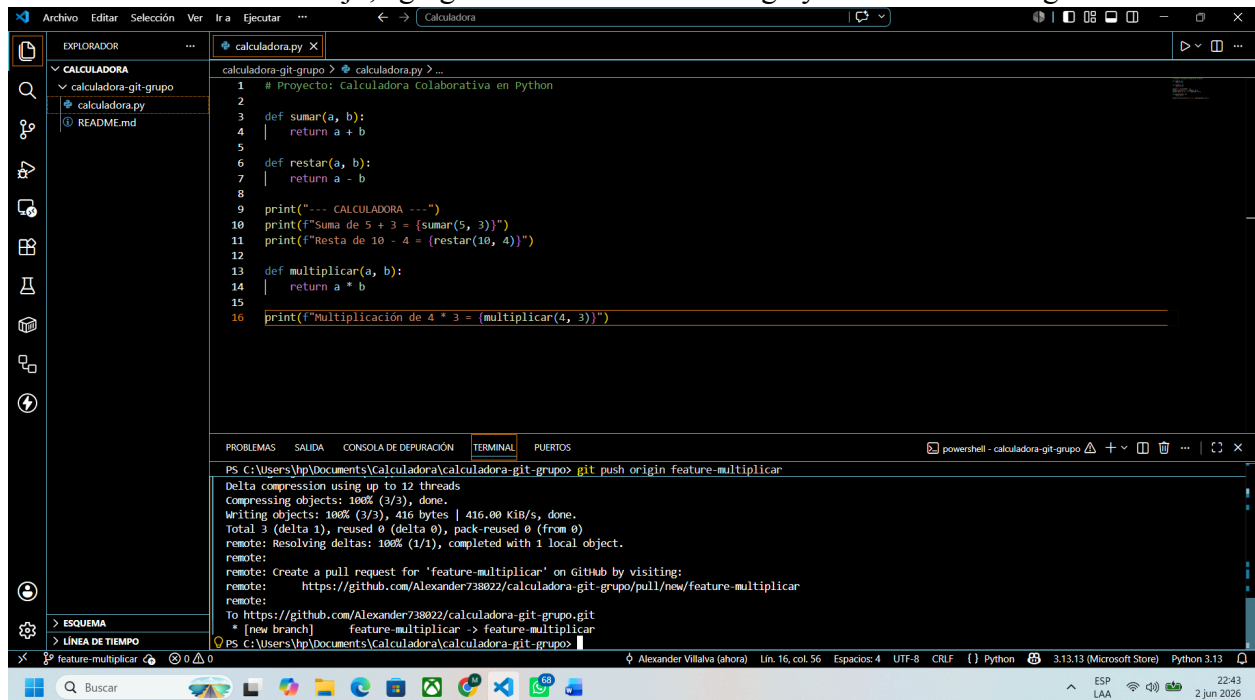




10: Regresar a la rama principal y descargar lo que se fusiona.



11: Crear la rama de trabajo, agregar nuevas líneas de código y subir cambios al github.

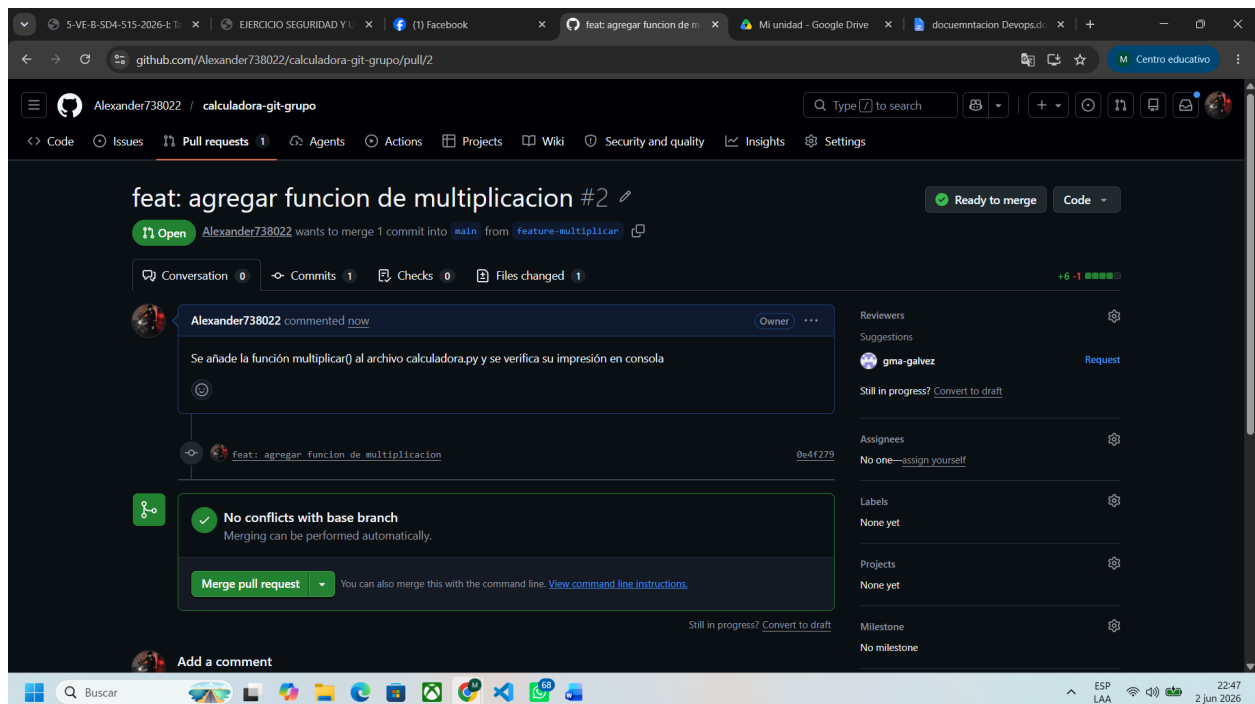


The screenshot shows a Windows IDE with a file explorer on the left showing a project named 'calculadora-git-grupo'. The main editor displays a Python file 'calculadora.py' with the following code:

```
1 # Proyecto: calculadora colaborativa en Python
2
3 def sumar(a, b):
4     return a + b
5
6 def restar(a, b):
7     return a - b
8
9 print("---- CALCULADORA ----")
10 print(f"Suma de 5 + 3 = {sumar(5, 3)}")
11 print(f"Resta de 10 - 4 = {restar(10, 4)}")
12
13 def multiplicar(a, b):
14     return a * b
15
16 print(f"Multiplicación de 4 * 3 = {multiplicar(4, 3)}")
```

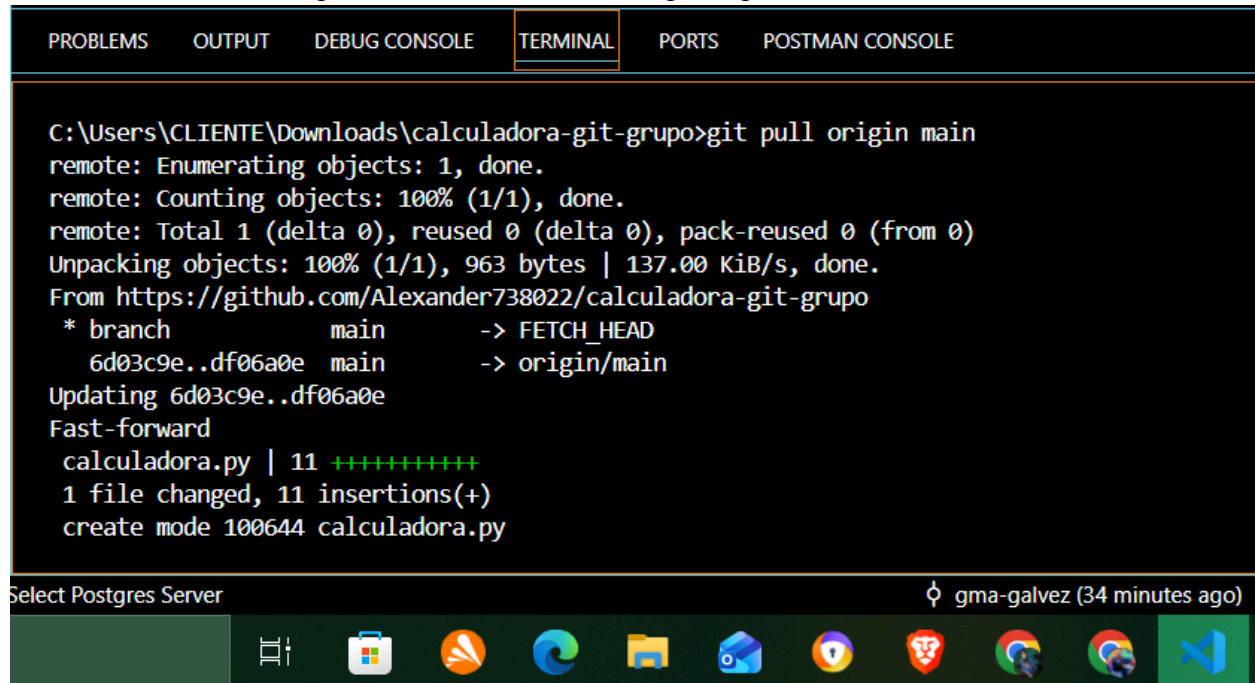
The terminal window at the bottom shows the execution of the following command: `git push origin feature-multiplicar`. The output indicates a successful push to the 'feature-multiplicar' branch on GitHub.

12: Creación de Pull Request.



(Estudiante B)

13. Sincronización del repositorio local con la rama principal

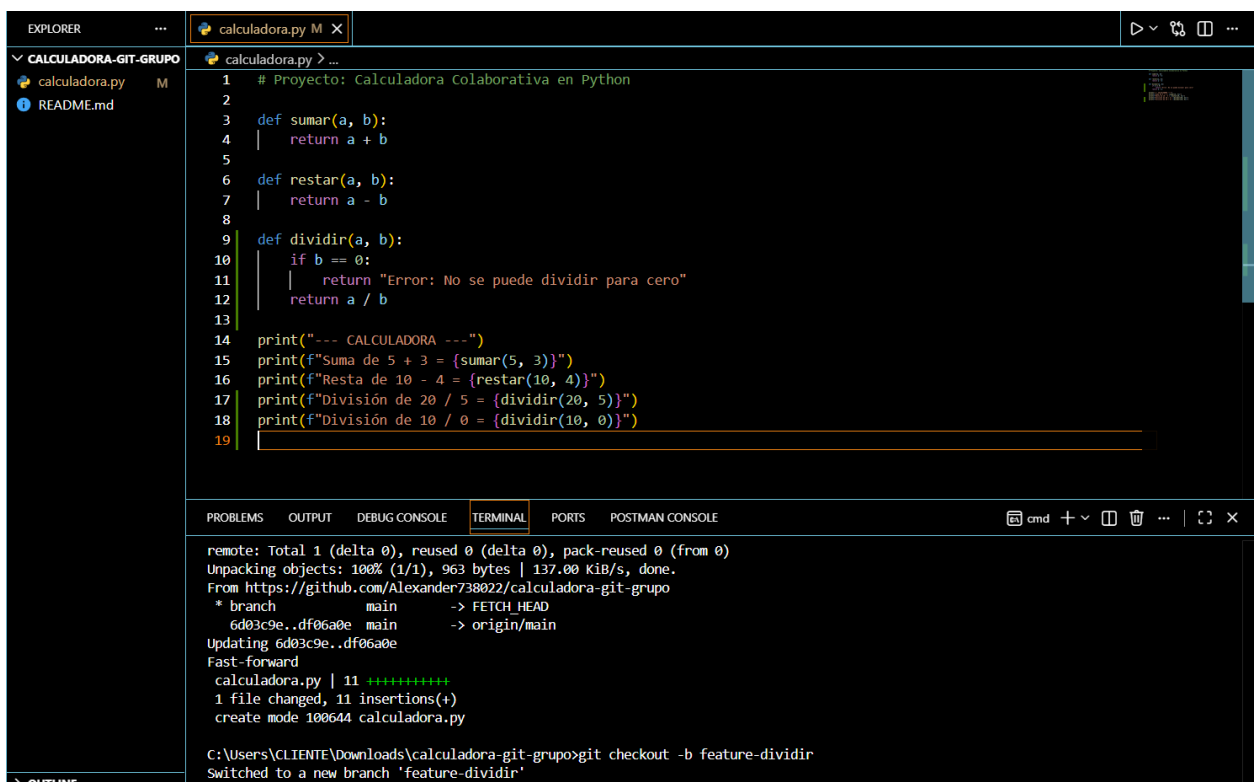


```
C:\Users\CLIENTE\Downloads\calculadora-git-grupo>git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 963 bytes | 137.00 KiB/s, done.
From https://github.com/Alexander738022/calculadora-git-grupo
* branch          main          -> FETCH_HEAD
   6d03c9e..df06a0e main        -> origin/main
Updating 6d03c9e..df06a0e
Fast-forward
 calculadora.py | 11 ++++++++
 1 file changed, 11 insertions(+)
 create mode 100644 calculadora.py
```

Explicación breve:

Se actualizó la rama principal local descargando los cambios fusionados en GitHub para mantener sincronizado el repositorio antes de continuar con el desarrollo colaborativo.

14. Creación de la rama feature-dividir e Implementación de la función de división

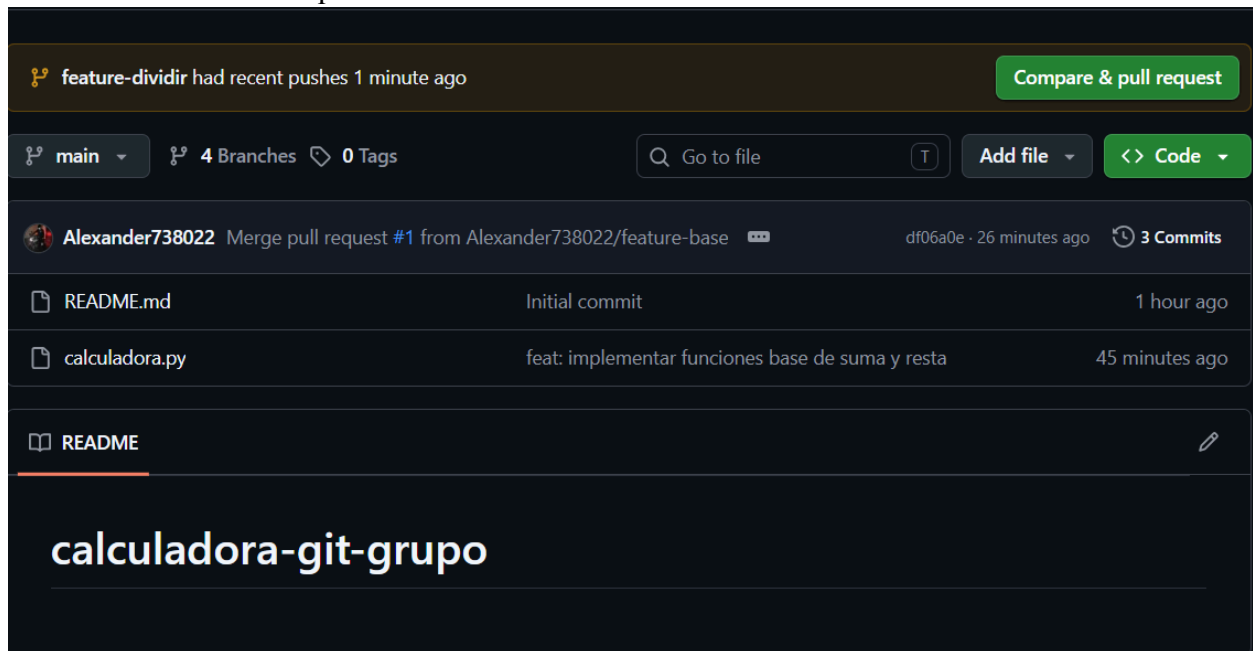


```
1 # Proyecto: Calculadora Colaborativa en Python
2
3 def sumar(a, b):
4     return a + b
5
6 def restar(a, b):
7     return a - b
8
9 def dividir(a, b):
10     if b == 0:
11         return "Error: No se puede dividir para cero"
12     return a / b
13
14 print("--- CALCULADORA ---")
15 print(f"Suma de 5 + 3 = {sumar(5, 3)}")
16 print(f"Resta de 10 - 4 = {restar(10, 4)}")
17 print(f"División de 20 / 5 = {dividir(20, 5)}")
18 print(f"División de 10 / 0 = {dividir(10, 0)}")
19
```

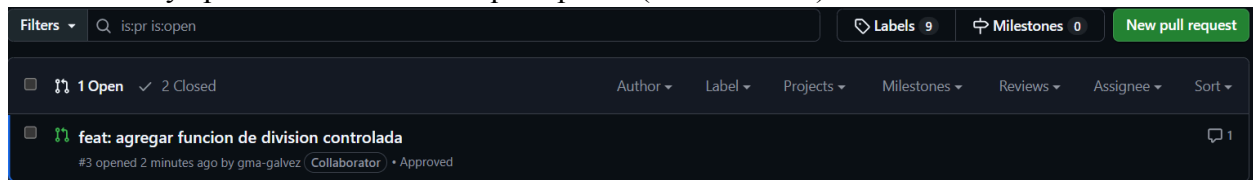
```
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 963 bytes | 137.00 KiB/s, done.
From https://github.com/Alexander738022/calculadora-git-grupo
* branch          main          -> FETCH_HEAD
   6d03c9e..df06a0e main        -> origin/main
Updating 6d03c9e..df06a0e
Fast-forward
 calculadora.py | 11 ++++++++
 1 file changed, 11 insertions(+)
 create mode 100644 calculadora.py

C:\Users\CLIENTE\Downloads\calculadora-git-grupo>git checkout -b feature-dividir
Switched to a new branch 'feature-dividir'
```

15. Creación del Pull Request de división



16. Revisión y aprobación del Pull Request por el (Estudiante A)



17: Provocación y Resolución del Conflicto de Integración

1. Crear y cambiarse a la nueva rama .

```
PS C:\Users\hp\Documents\Calculadora\calculadora-git-grupo> git checkout -b cambio-bienvenida-A
Switched to a new branch 'cambio-bienvenida-A'
PS C:\Users\hp\Documents\Calculadora\calculadora-git-grupo>
```

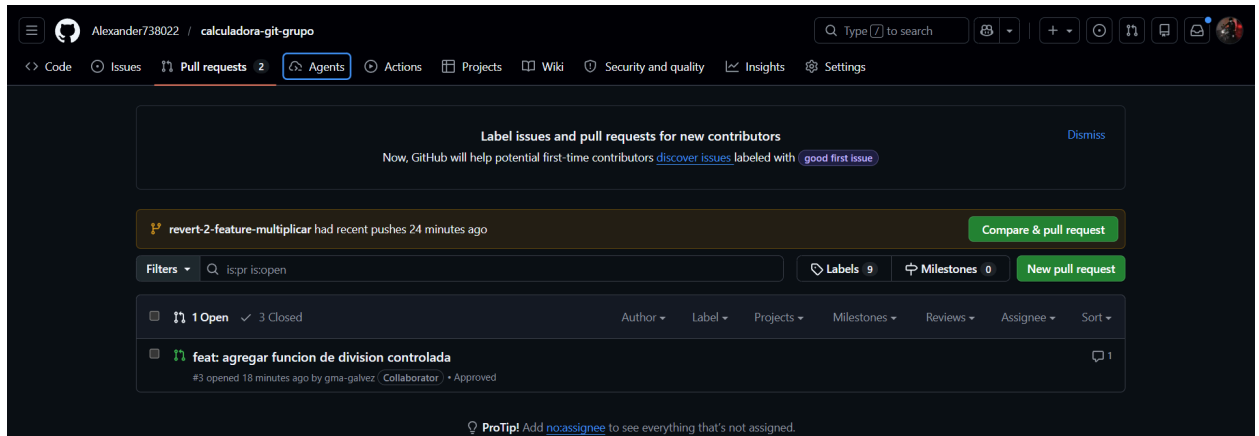
2. Modificar el archivo .



3. Guardar en Git y subir a GitHub.

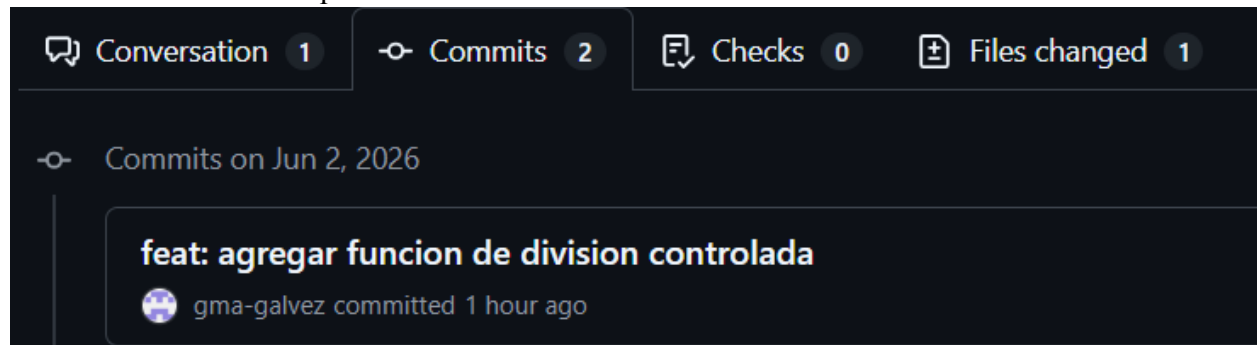
```
PS C:\Users\hp\Documents\Calculadora\calculadora-git-grupo> git checkout -b cambio-bienvenida-A
Switched to a new branch 'cambio-bienvenida-A'
PS C:\Users\hp\Documents\Calculadora\calculadora-git-grupo> git add calculadora.py
PS C:\Users\hp\Documents\Calculadora\calculadora-git-grupo> git commit -m "style: renombrar encabezado con identidad de A"
[cambio-bienvenida-A 349d26f] style: renombrar encabezado con identidad de A
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\hp\Documents\Calculadora\calculadora-git-grupo> git push origin cambio-bienvenida-A
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 389 bytes | 389.00 KiB/s, done.
```

4. El Estudiante A abre el Pull Request en GitHub y **él mismo se aprueba y se hace el Merge** a la rama **main** desde la web. Ahora el repositorio remoto tiene ese título modificado.

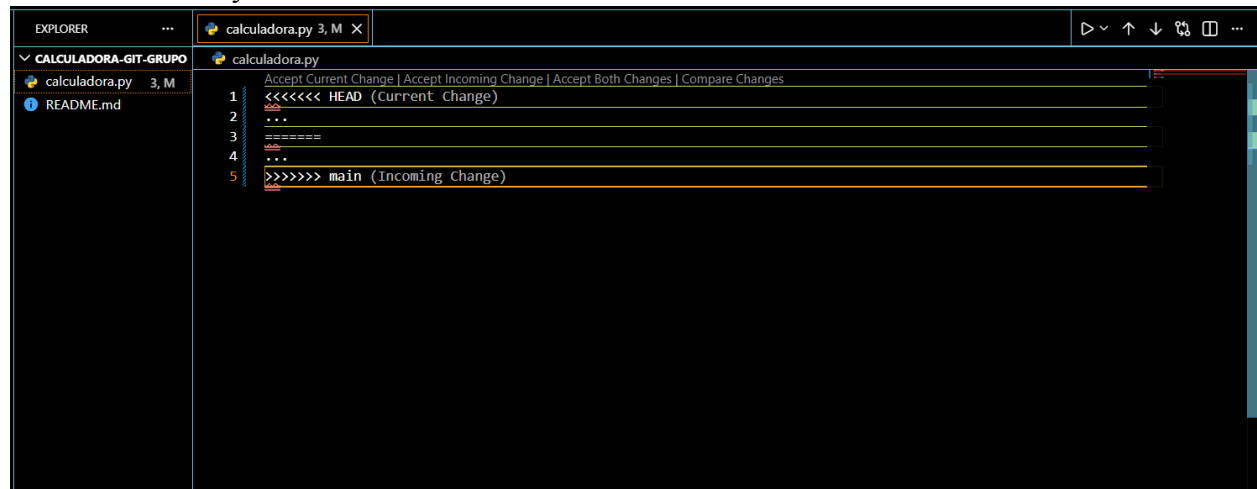


(Estudiante B)

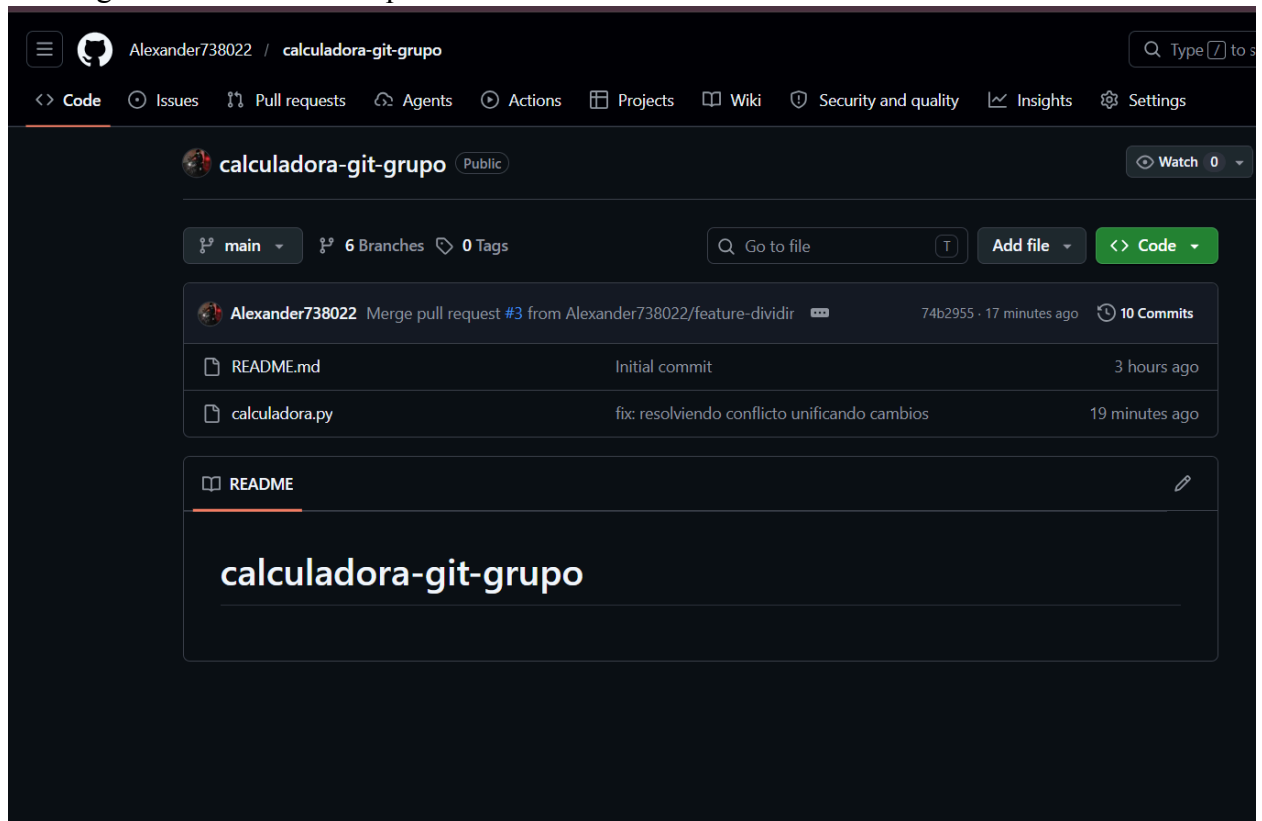
18.Creación del Pull Request



19.Identificación y resolución del conflicto en Visual Studio Code

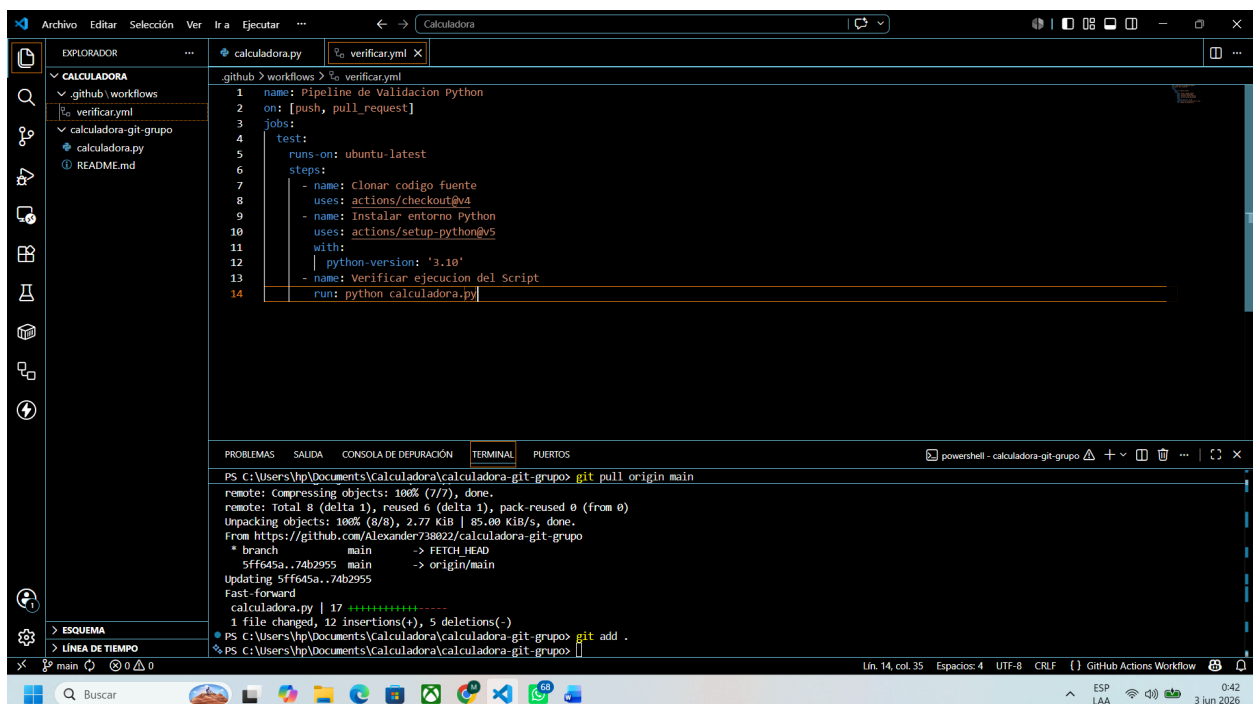


20. Merge exitoso del Pull Request

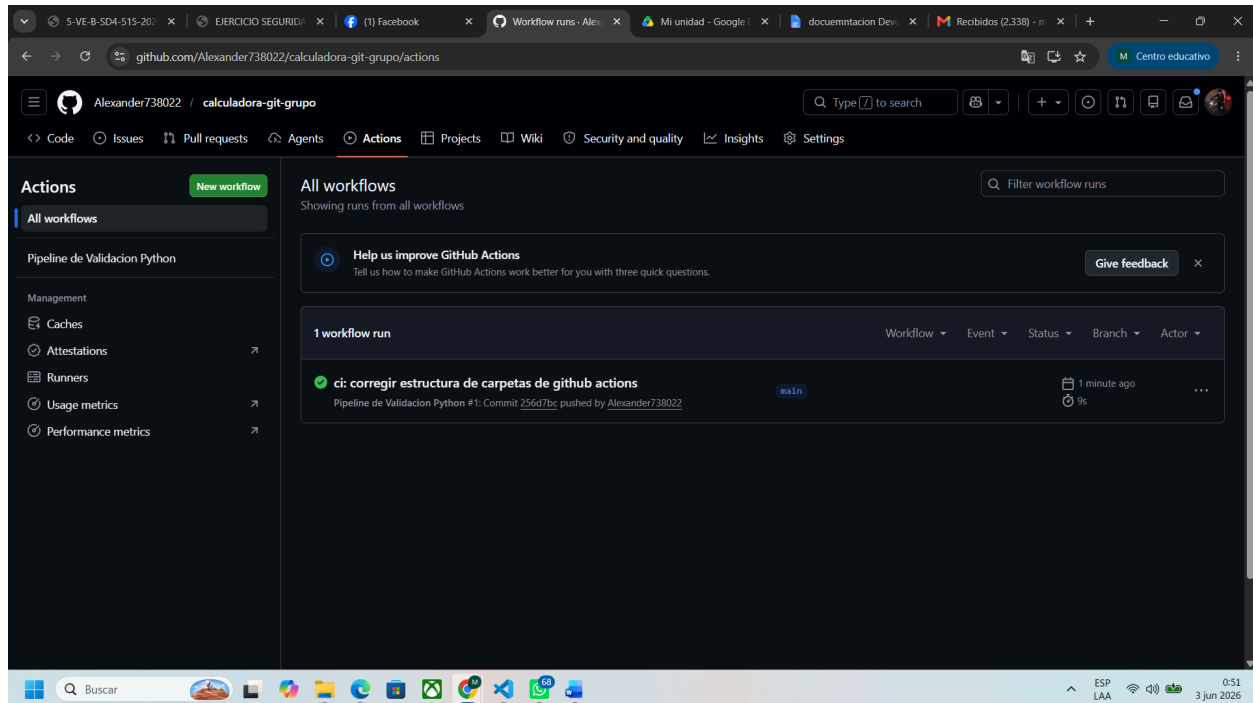


21: Configuración de la Automatización.

Creación de las carpetas .github, dentro de la misma le ponemos el workflows y finalmente un documento llamado verificar.yml.



22: Check verde de GitHub Actions exitoso.



CONCLUSIONES.

(Estudiante A)

La práctica de laboratorio demostró con éxito la viabilidad de la metodología DevOps mediante un flujo de trabajo colaborativo y unificado en Git, donde se resolvieron conflictos de integración de forma manual y controlada desde el entorno local. Asimismo, la implementación de un *pipeline* automatizado a través de GitHub Actions validó la correcta ejecución del script en Python, garantizando los principios de Integración Continua (CI) y asegurando que el software final sea robusto, libre de errores de sintaxis y esté listo para un entorno de despliegue profesional.

(Estudiante B)

La práctica permitió comprender el funcionamiento del trabajo colaborativo mediante Git y GitHub. Se desarrollaron habilidades para crear ramas, realizar commits, generar Pull Requests y resolver conflictos de integración. También se observó la importancia de la revisión y aprobación de cambios antes de incorporarlos al proyecto principal. Estos conocimientos son esenciales para participar en equipos de desarrollo de software modernos.